

Stratégie de test : couches, Testcontainers et portes de couverture

[TESTING.md](#) organise les tests de DataShare en **couches** : chacune a un périmètre, un outil, des dépendances et un job CI (Continuous Integration) dédiés. L'enjeu est de placer chaque vérification au bon niveau : vite et isolé pour une règle de gestion, réaliste pour un endpoint, bout en bout pour un parcours utilisateur. Le détail MockMvc / RestAssured est dans [tests-back-mockmvc-restassured-randomport.md](#).

Les couches

Couche	Périmètre	Outil	Dépendances	Job CI
Back unitaire	services, règles isolées	JUnit 5 + Mockito	aucune	backend-unit
Back intégration	repositories, contrôleurs (MockMvc), sécurité, migrations Flyway	Spring Boot Test + Testcontainers	PostgreSQL, MinIO (conteneurs jetables)	backend-integ
Back fonctionnel	scénarios API enchaînant des requêtes	MockMvc + Testcontainers	idem	backend-integ
Front unitaire	pipes, services, guards, logique isolée	Vitest	aucune	frontend-unit
Front intégration	composant + template + DI, intercepteur, formulaire	Vitest + TestBed + jsdom	HttpTestingController (réponses simulées)	frontend-integ
Front e2e (end-to-end)	3-4 parcours critiques	Cypress	PostgreSQL + MinIO + backend réel + ng serve	frontend-e2e

Ce n'est pas une pyramide à 4 étages : le back a **3 couches** (unitaire, intégration, fonctionnel), le front **3 couches** (unitaire, intégration, e2e), soit 6 lignes de stratégie plus une garde de bundle (`assert-prod-bundle` , vérifie par `grep` que la config debug ne fuit pas en production).

Règle de placement

- une **règle de gestion** → test unitaire ;
- un **endpoint** → test d'intégration (statut, corps, autorisation, erreurs) ;
- un **parcours utilisateur critique** → test e2e ;
- le reste (rendu, câblage DI) → intégration front.

Testcontainers plutôt que des mocks

Testcontainers démarre de vrais conteneurs Docker jetables pendant le test :

```

@Bean
@ServiceConnection
PostgreSQLContainer postgresContainer() {
    return new PostgreSQLContainer(DockerImageName.parse("postgres:16-alpine"));
}

```

`@ServiceConnection` branche automatiquement l'URL de la base du conteneur dans Spring.

	Mock (Mockito)	Testcontainers
Ce qu'on vérifie	la logique de la classe, sans I/O	le comportement réel avec la vraie techno
Repository	on suppose que la requête est correcte	la requête, le schéma et les migrations Flyway s'exécutent vraiment
Coût	quelques ms	secondes, Docker requis
Où dans DataShare	<code>backend-unit</code> (services)	<code>backend-integ</code> (repositories, contrôleurs, sécurité)

Justification (déduite de la répartition de `TESTING.md`, non écrite telle quelle dans le document) : les tests d'intégration vérifient des choses qu'un mock ne peut pas prouver : les migrations Flyway, les requêtes JPA (par exemple `findByIdAndOwnerId`), les codes HTTP réels et l'objet réellement présent dans le bucket MinIO (`FileControllerIT`). Un mock de repository renverrait ce qu'on lui dit de renvoyer. Les mocks restent utiles là où il n'y a pas d'infrastructure à valider : les règles du service.

Portes de couverture à 70 %

- Objectif de **70 %** de lignes, exigence du cahier des charges.
- **Back** : JaCoCo (Java Code Coverage). `jacoco:merge` fusionne unit et integ, puis `jacoco:check` impose `COVEREDRATIO ≥ 0.70` (BUNDLE, lignes) au `verify`, donc dans `backend-integ`. Exclusions : `config/**`, `*Application`, `**/dto/**`, `*Properties`, `PurgeRunner`.
- **Front** : Vitest + coverage-v8, puis `tools/check-coverage.mjs` (`THRESHOLD = 70`) dans `frontend-integ`.
- Ce sont des **portes bloquantes** : sous 70 %, le job échoue et la PR ne merge pas.
- Couverture actuelle déclarée dans `TESTING.md` : ~88 % back, ~95 % front (chiffres du document, non recalculés).

Fusionner unit et integ compte ce que chaque niveau couvre ; mesurer les unitaires seuls sous-estimerait le back, dont les contrôleurs ne sont touchés que par l'intégration.

Dans DataShare

- Stratégie et couverture par US : `TESTING.md` §1, §2, §6.
- Conteneur PostgreSQL : `TestcontainersConfiguration.java` ; MinIO : `MinioTestcontainersConfiguration.java`.
- Bases d'IT : `AbstractIntegrationTest.java`.
- Porte back : `pom.xml` (`jacoco:check`, minimum 0.70).
- Porte front : `check-coverage.mjs`.
- Critère de « fait » : tests unit + integ verts, e2e si parcours critique, doc à jour, CI verte.

À retenir

1. Chaque couche a son outil et son job CI : règle de gestion → unitaire, endpoint → intégration, parcours critique → e2e.
2. Testcontainers exécute la vraie base et le vrai stockage : il prouve ce qu'un mock ne peut pas (migrations, requêtes, codes HTTP réels).
3. Les mocks restent pour les services isolés : rapides, sans Docker.
4. La couverture de 70 % de lignes est une porte bloquante en CI, back (JaCoCo, unit + integ fusionnés) et front (Vitest).
5. Le seuil est un plancher, pas une preuve de qualité : la couverture actuelle dépasse largement 70 %.